

Multi-Agent Deep Reinforcement Learning Applied to a Tennis Environment

Catarina Pires

Deep Reinforcement Learning Nanodegree, Udacity

Abstract—This project applies Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [1] to a cooperative tennis environment in which two agents must keep a ball in play over a net. MADDPG extends DDPG [2] to multi-agent settings by pairing centralised critics with decentralised actors, allowing agents to coordinate during training while relying only on local observations at execution time. The environment, which requires an average score of +0.5 over 100 consecutive episodes, was successfully solved in 2180 episodes.

I. INTRODUCTION

Reinforcement learning (RL) has emerged as a powerful framework for training agents to solve complex decision-making problems through interaction with an environment. However, many of its most prominent successes have been achieved in single-agent domains, where modelling the behaviour of other actors in the environment is unnecessary. In contrast, many real-world problems involve multiple interacting agents, where non-stationarity, coordination challenges, and emergent behaviours arise as agents learn simultaneously. These challenges motivate the development of multi-agent reinforcement learning algorithms capable of learning coordinated policies in such environments.

One approach for addressing these challenges is the Multi-Agent Deep Deterministic Policy Gradient (MADDPG) algorithm proposed by Lowe et al. [1]. MADDPG extends the Deep Deterministic Policy Gradient (DDPG) algorithm [2] to multi-agent environments by introducing centralized critics during training while maintaining decentralized actors for execution. This framework allows agents to leverage additional information about other agents during learning while still operating using only local observations at test time, helping to mitigate the non-stationarity inherent in multi-agent learning.

In this project, MADDPG is applied to a cooperative tennis environment in which two agents must learn to keep a ball in play over a net. The environment is described in detail in Section III.

II. METHODS

A. Multi-Agent Deep Deterministic Policy Gradient

Multi-Agent Deep Deterministic Policy Gradient (MADDPG) is an extension of DDPG designed for multi-agent environments where several agents learn simultaneously. In such settings, the environment becomes non-stationary from the perspective of each agent because the policies of other agents are also changing during training. MADDPG

addresses this challenge by combining decentralized policy learning with centralized value estimation.

Key Components: MADDPG uses an actor-critic architecture similar to DDPG, but with separate actors for each agent and critics that can condition on information from all agents:

- Actor network $\mu_i(o_i|\theta^{\mu_i})$: Determines the action of agent i based on its local observation o_i .
- Critic network $Q_i(x, a_1, \dots, a_N|\theta^{Q_i})$: Estimates the Q-value for agent i using the global state x and the actions of all agents.
- Target actor networks $\mu'_i(o_i|\theta^{\mu'_i})$: Slowly tracking copies of each actor.
- Target critic networks $Q'_i(x, a_1, \dots, a_N|\theta^{Q'_i})$: Slowly tracking copies of each critic.

Core Mechanisms: MADDPG mitigates the non-stationarity of multi-agent environments by using centralized critics during training. While each agent executes actions using only its local observations (decentralized execution), the critic networks have access to the observations and actions of all agents during training. This allows the critics to better evaluate the joint action space and provide more stable learning signals to the actors. As in DDPG, experience replay and target networks are used to stabilize training.

Training Process: Each agent maintains its own actor and critic networks. During training, agents collect joint experiences of the form $(x, a_1, \dots, a_N, r_1, \dots, r_N, x')$ and store them in a shared replay buffer. The critic for each agent is updated by minimizing a temporal-difference loss based on the Bellman equation, while the actor is updated using the deterministic policy gradient computed through the centralized critic. Target networks are updated using soft updates with a small parameter τ to ensure stable learning.

The algorithm alternates between collecting experience in the environment and updating the networks from sampled transitions. The centralized critic enables each agent to learn in the presence of other learning agents, while decentralized actors allow the learned policies to be executed using only local observations.

Algorithm 1 Multi-Agent Deep Deterministic Policy Gradient (MADDPG)

```

1: Initialize actor networks  $\mu_i(o_i|\theta^{\mu_i})$  and critic networks
    $Q_i(x, a_1, \dots, a_N|\theta^{Q_i})$  for each agent  $i$ 
2: Initialize target networks  $\mu'_i$  and  $Q'_i$  with weights  $\theta^{\mu'_i} \leftarrow$ 
    $\theta^{\mu_i}$ ,  $\theta^{Q'_i} \leftarrow \theta^{Q_i}$ 
3: Initialize shared replay buffer  $\mathcal{R}$ 
4: for episode = 1,  $M$  do
5:   Initialize random processes  $\mathcal{N}_i$  for action exploration
6:   Receive initial observations  $(o_1, \dots, o_N)$ 
7:   for  $t = 1, T$  do
8:     for each agent  $i$  do
9:       Select action  $a_i = \mu_i(o_i|\theta^{\mu_i}) + \mathcal{N}_i$ 
10:    end for
11:    Execute joint action  $(a_1, \dots, a_N)$ 
12:    Observe rewards  $(r_1, \dots, r_N)$  and next observa-
13:    tions  $(o'_1, \dots, o'_N)$ 
14:    Store transition  $(x, a_1, \dots, a_N, r_1, \dots, r_N, x')$  in
        $\mathcal{R}$ 
15:    if  $|\mathcal{R}| > \text{batch\_size}$  then
16:      Sample minibatch of  $N$  transitions from  $\mathcal{R}$ 
17:      for each agent  $i$  do
18:        Set target:
19:         $y_i = r_i + \gamma Q'_i(x', a'_1, \dots, a'_N)$ 
20:        where  $a'_j = \mu'_j(o'_j)$ 
21:        Update critic by minimizing:
22:         $L = \frac{1}{N} \sum (y_i - Q_i(x, a_1, \dots, a_N))^2$ 
23:        Update actor using policy gradient:
24:         $\nabla_{\theta^{\mu_i}} J$ 
25:         $\frac{1}{N} \sum \nabla_{a_i} Q_i(x, a_1, \dots, a_N) \nabla_{\theta^{\mu_i}} \mu_i(o_i)$ 
26:      end for
27:      for each agent  $i$  do
28:        Soft update target networks:
29:         $\theta^{Q'_i} \leftarrow \tau \theta^{Q_i} + (1 - \tau) \theta^{Q'_i}$ 
30:         $\theta^{\mu'_i} \leftarrow \tau \theta^{\mu_i} + (1 - \tau) \theta^{\mu'_i}$ 
31:      end for
32:    end if
33:     $o_i \leftarrow o'_i$ 
34:  end for
35: end for

```

III. ENVIRONMENT

The Unity Machine Learning Agents (ML-Agents) toolkit [3] is an open-source Unity plugin that enables games and simulations to serve as environments for training intelligent agents. This project makes use of one of Unity’s built-in environments to train and evaluate the MADDPG algorithm.

As shown in Figure ??, two agents control rackets to bounce a ball over a net in a cooperative tennis setting. Each agent receives a reward of +0.1 for hitting the ball over the net, and a penalty of -0.01 if the ball hits the ground or goes out of bounds. The goal of each agent is therefore to

keep the ball in play for as long as possible, making this a cooperative task in which both agents must coordinate to maintain rallies.

The observation space consists of 8 continuous variables representing the position and velocity of the ball and racket. Each agent receives only its own local observation, with no direct access to the other agent’s state. Each agent selects from two continuous actions corresponding to movement toward or away from the net, and jumping. The task is episodic, and the environment is considered solved when the agents achieve a mean score of at least +0.5 over 100 consecutive episodes, where the score for each episode is taken as the maximum score obtained by either agent.

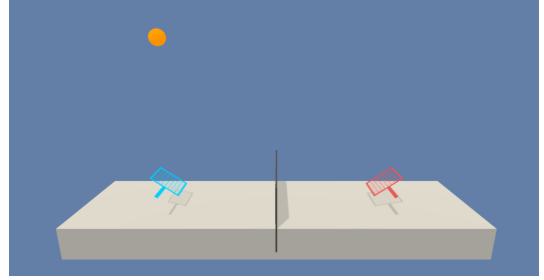


Figure 1. Environment

IV. IMPLEMENTATION DETAILS

The MADDPG algorithm was implemented using the PyTorch deep learning framework. Each agent maintains its own actor and critic networks along with corresponding target networks, following the centralized training and decentralized execution paradigm. During execution, each actor selects actions based solely on its local observations, while the critics are trained using the joint observations and actions of all agents. Experiences are stored in a shared replay buffer with a capacity of 5×10^5 transitions, and minibatches of size 128 are sampled during training.

Network Architecture: Both the actor and critic networks are implemented as fully connected feed-forward neural networks. The actor network maps local observations to continuous actions using three hidden layers of sizes 128, 64, and 32 with ReLU activations, followed by a tanh output layer to bound the action values. The critic network receives the concatenated states of all agents and processes them through two fully connected layers before combining them with the joint action vector. The resulting representation is passed through an additional hidden layer to estimate the Q-value of the joint state-action pair.

Training Procedure: Training follows the standard MADDPG approach using off-policy learning with experience replay. The critic networks are optimized using the Adam optimizer with a learning rate of 10^{-3} , while the actor networks use a learning rate of 1.9×10^{-4} . A discount factor of $\gamma = 0.99$ is used to compute target Q-values, and target

networks are updated using soft updates with $\tau = 10^{-3}$. Gradient clipping with a maximum norm of 1.0 is applied to improve training stability.

Exploration and Optimization: Exploration is achieved by adding noise to the actions selected by the actors, controlled by an exploration parameter ϵ initialized to 1.0 and gradually decayed during training. Learning updates are performed every 5 environment steps, with 10 gradient update passes per learning phase. L2 weight decay is applied to both actor and critic networks to reduce overfitting. To ensure reproducibility, a fixed random seed is used across NumPy, PyTorch, and Python’s random module.

The main hyperparameters and training settings are summarized in Table I.

Parameter	Value	Description
Replay buffer size	5×10^5	Maximum number of stored experiences
Batch size	128	Number of experiences sampled per update
Discount factor (γ)	0.99	Future reward discount
Soft update factor (τ)	1×10^{-3}	Rate for updating target networks
Actor learning rate	1.9×10^{-4}	Adam optimizer learning rate for actor
Critic learning rate	1×10^{-3}	Adam optimizer learning rate for critic
Update frequency	5 steps	Number of environment steps between updates
Learning passes per update	10	Number of gradient updates performed at each learning step
Gradient clipping	1.0	Maximum gradient norm for critic updates
OU noise (θ)	0.15	Mean reversion rate for Ornstein-Uhlenbeck noise
OU noise (σ)	0.2	Volatility parameter for Ornstein-Uhlenbeck noise
OU noise (ϵ)	1.0	explore->exploit noise process added to act step
OU noise (ϵ decay)	1×10^{-6}	decay rate for noise process

Table I. Hyperparameters and training settings used for the MADDPG agent.

V. RESULTS

The following results, Figure 2, demonstrate the successful application of MADDPG to the multi-agent collaboration and competition task. The algorithm achieved the target performance criterion of an average score of +0.5 over 100 consecutive episodes in 2180 training episodes.

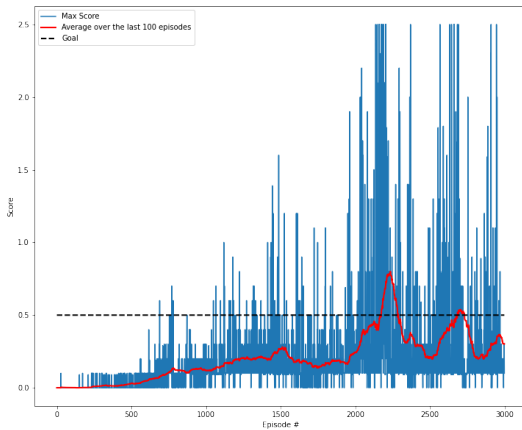


Figure 2. Results

VI. CONCLUSION AND FUTURE WORK

This project successfully applied MADDPG [1] to a cooperative multi-agent tennis task, reaching the target average

score of +0.5 over 100 episodes in 2180 training episodes. MADDPG was chosen for its ability to handle continuous action spaces and its centralised training, decentralised execution framework — each agent’s critic has access to all agents’ observations and actions during training, providing a richer learning signal, while actors rely only on local observations at deployment.

Several improvements remain worth exploring as discussed in the sections 4.2 and 4.3 of [1]. The first has each agent learn its own approximation of other agents’ policies, which removes the need for centralised training. The second uses policy ensembles to boost performance, though at the cost of greater compute during both training and deployment. Eliminating the centralised training requirement is especially valuable in settings with many agents, as the centralised critic’s input space — and therefore its parameter count — would otherwise grow exponentially with the number of agents.

Furthermore, given DDPG’s well-documented training instability [4], algorithms such as TRPO [5] or PPO [6] may offer better stability without the need for experience replay, making them naturally suited to decentralised multi-agent settings.

REFERENCES

- [1] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” *Advances in neural information processing systems*, vol. 30, 2017.
- [2] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” *arXiv preprint arXiv:1509.02971*, 2015.
- [3] “GitHub - Unity-Technologies/ml-agents: The Unity Machine Learning Agents Toolkit (ML-Agents) is an open-source project that enables games and simulations to serve as environments for training intelligent agents using deep reinforcement learning and imitation learning. — github.com,” <https://github.com/Unity-Technologies/ml-agents>, [Accessed 30-05-2024].
- [4] G. Matheron, N. Perrin, and O. Sigaud, “The problem with ddpg: understanding failures in deterministic environments with sparse rewards,” *arXiv preprint arXiv:1911.11679*, 2019.
- [5] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz, “Trust region policy optimization,” in *International conference on machine learning*. PMLR, 2015, pp. 1889–1897.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.